

Laboratório de Sistemas Digitais

Aula Teórico-Prática 12

Ano Letivo 2024/25

As construções “**for...generate**” e
“**if...generate**” em VHDL
Atributos pré-definidos em VHDL

Conteúdo

- A construção **for...generate** de VHDL para replicação de circuitos lógicos
 - Exemplos com
 - Instanciação de entidades
 - Atribuições concorrentes (condicionais)
 - Processos
- Atributos pré-definidos em VHDL

A Construção **for...generate** de VHDL para Replicação de Hardware – Ciclo Estrutural

```
label : for <index> in <KMIN> to <KMAX> generate
```

<circuito lógico a replicar descrito com

atribuição(ões) concorrente(s) condicional(is)

processos(s)

instanciação de entidade(s)>

```
end generate;
```

em que:

<index> - índice inteiro (com declaração implícita)

<KMIN> e <KMAX> – constantes inteiras (valor definido em *compile time*)

A construção **for...generate** deve ser escrita no corpo de uma arquitetura (e fora de processos!)

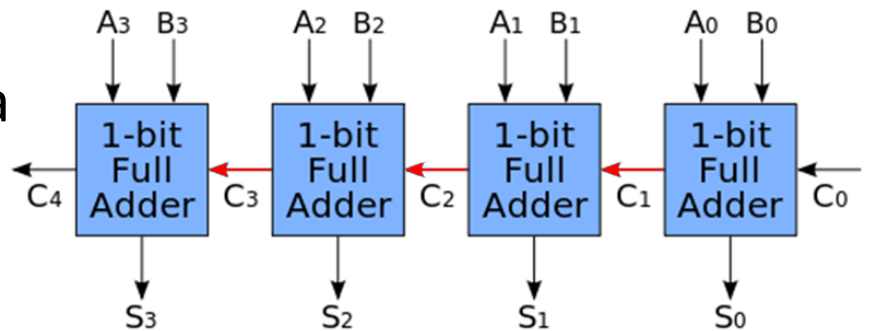
Também pode ser escrita na forma:

```
label : for <index> in <KMAX> downto <KMIN>
```

A Construção **for...generate** de VHDL

Exemplo de Motivação

- Somador de 4 bits (guião prático 3)
- Construído com 4 somadores completos de 1 bit em cascata
- Esqueleto do somador completo (retirado do guião)



```
library IEEE;
use IEEE.STD_LOGIC_1164.all;

entity FullAdder is
    port(a, b, cin : in  std_logic;
          s, cout   : out std_logic);
end FullAdder;
```

```
architecture Behavioral of FullAdder is
begin
```

```
    -- Especifique aqui as equações lógicas para as saídas "s" e "cout"
end Behavioral;
```

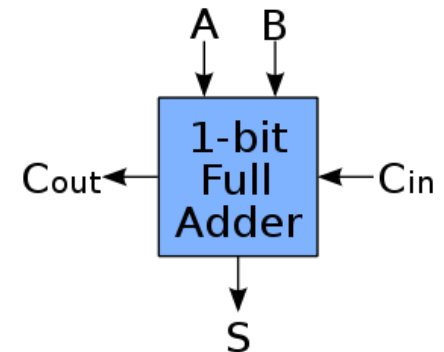
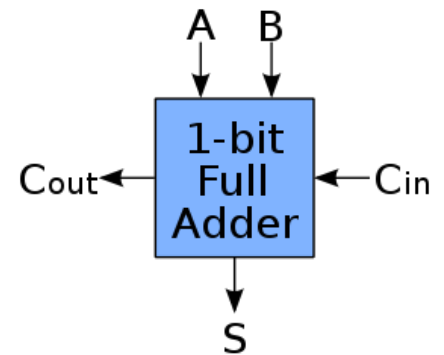


Figura 2 – Esqueleto do código VHDL da entidade **FullAdder** e respetiva arquitetura **Behavioral**.

Código Completo de Somador Completo de 1 bit

```
Text Editor - C:/Users/asoliveira/CloudStation/LSDig2017/SlidesTP/Gen...
File Edit View Project Processing Tools Window Help Search altera.com
[Icons] 267 268
1 library IEEE;
2 use IEEE.STD_LOGIC_1164.all;
3
4 entity FullAdder is
5     port(a, b, cin : in std_logic;
6         s, cout    : out std_logic);
7 end FullAdder;
8
9 architecture Behavioral of FullAdder is
10 begin
11     s    <= a xor b xor cin;
12     cout <= (a and b) or (a and cin) or (b and cin);
13 end Behavioral;
14
```



A Construção **for...generate** de VHDL

Exemplo de Motivação

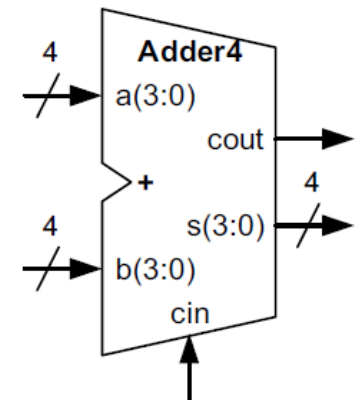
- Esqueleto do somador de 4 bits – cascata (retirado do guião)

```
-- Inclua as bibliotecas e os pacotes necessários

entity Adder4 is
    port(a, b : in  std_logic_vector(3 downto 0);
          cin : in  std_logic;
          s   : out std_logic_vector(3 downto 0);
          cout : out std_logic);
end Adder4;

architecture Structural of Adder4 is
    -- Declare um sinal interno (carryOut) do tipo std_logic_vector (de
    -- C bits) que interligará os bits de carry dos somadores entre si
begin
    bit0: entity work.FullAdder(Behavioral)
        port map(a    => a(0),
                 b    => b(0),
                 cin  => cin,
                 s    => s(0),
                 cout => carryOut(0));

    -- complete para os restantes bits (1 a 3)
end Structural;
```

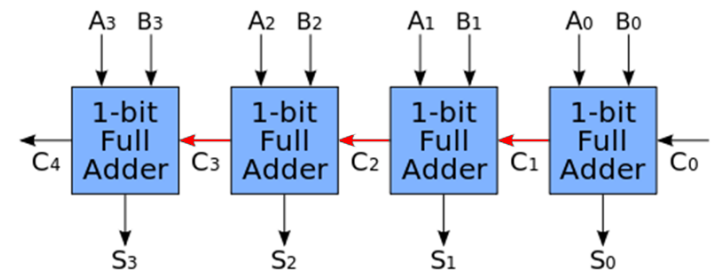


E se o somador
tivesse 64 bits?
E se fosse
parametrizável
(estaticamente /
em *compile time*)?

Figura 4 – Esqueleto do código VHDL da entidade **Adder4** e arquitetura **Structural**.

Instanciação de 4 Somadores Completos de 1 bit com **for...generate**

```
Text Editor - C:/Users/asoliveira/CloudStation/LSDig2017/SlidesTP/...
File Edit View Project Processing Tools Window Help Search altera.com
267 268
1 library IEEE;
2 use IEEE.STD_LOGIC_1164.all;
3
4 entity Adder4 is
5     port(a, b : in std_logic_vector(3 downto 0);
6         cin : in std_logic;
7         s : out std_logic_vector(3 downto 0);
8         cout : out std_logic);
9 end Adder4;
10
11 architecture Structural of Adder4 is
12     signal s_carry : std_logic_vector(4 downto 0);
13
14 begin
15     s_carry(0) <= cin;
16
17     adder_loop : for i in 0 to 3 generate
18     cell: entity work.FullAdder(Behavioral)
19     port map(a => a(i),
20             b => b(i),
21             cin => s_carry(i),
22             s => s(i),
23             cout => s_carry(i+1));
24
25     end generate;
26
27     cout <= s_carry(4);
28 end Structural;
29
```



Construção de um Somador Parametrizável de N bits com **for...generate**

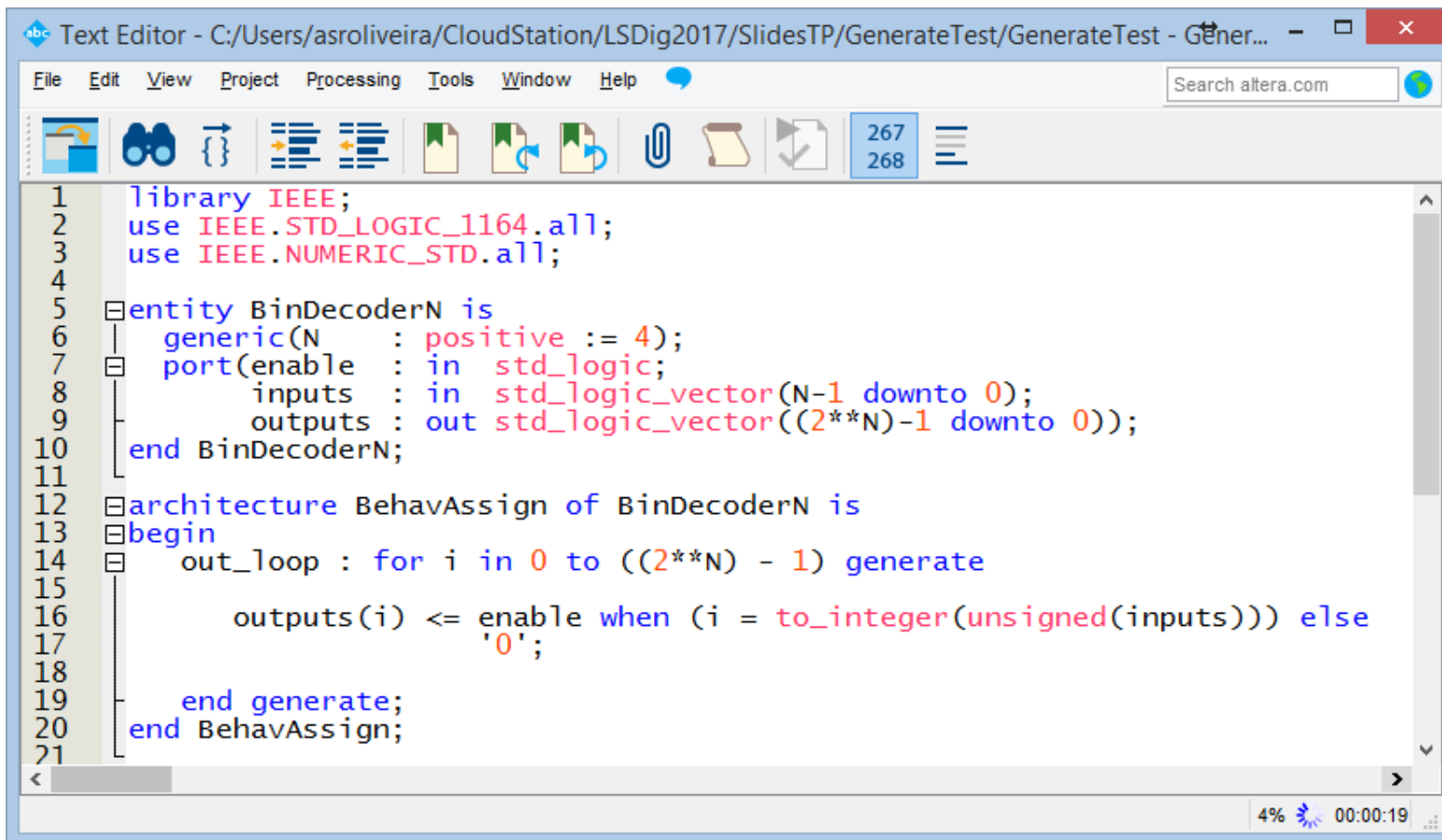
```
Text Editor - C:/Users/asoliveira/CloudStation/LSDig2017/SlidesTP/Generate...
File Edit View Project Processing Tools Window Help Search altera.com
267 268

1  library IEEE;
2  use IEEE.STD_LOGIC_1164.all;
3
4  entity AdderN is
5      generic(N : integer := 8);
6      port(a, b : in std_logic_vector(N-1 downto 0);
7            cin : in std_logic;
8            s : out std_logic_vector(N-1 downto 0);
9            cout : out std_logic);
10 end AdderN;
11
12 architecture Structural of AdderN is
13
14     signal s_carry : std_logic_vector(N downto 0);
15
16 begin
17     s_carry(0) <= cin;
18
19     adder_loop : for i in 0 to (N-1) generate
20     cell: entity work.FullAdder(Behavioral)
21     port map(a => a(i),
22             b => b(i),
23             cin => s_carry(i),
24             s => s(i),
25             cout => s_carry(i+1));
26     end generate;
27
28     cout <= s_carry(N);
29 end Structural;
30
```

Ln 1 Col 1 VHDL File 100% 00:00:21

Exemplo de **for...generate** com uma Atribuição Condicional

- Decodificador binário de $N \rightarrow 2^N$ bits, com N parametrizável

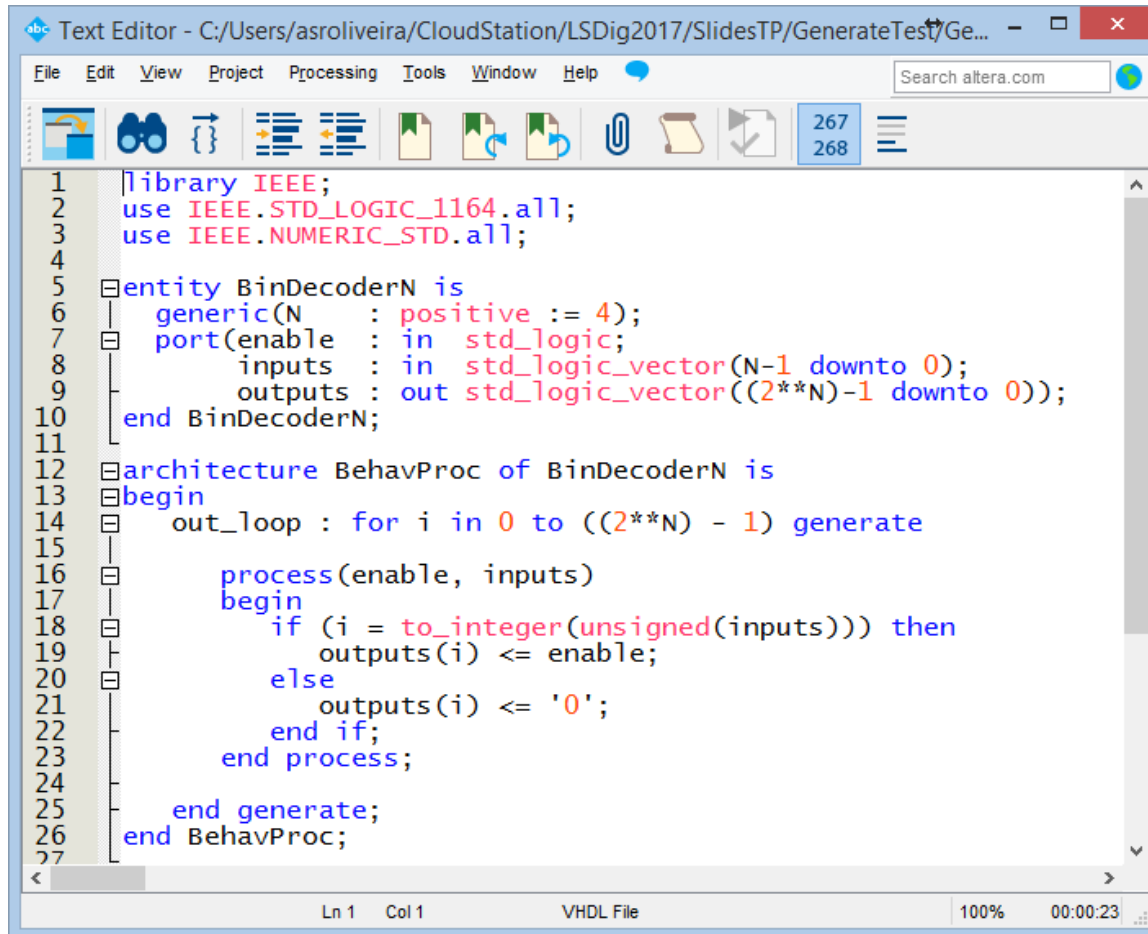


The screenshot shows a text editor window titled "Text Editor - C:/Users/asroliveira/CloudStation/LSDig2017/SlidesTP/GenerateTest/GenerateTest - Gener...". The editor contains VHDL code for a binary decoder entity named BinDecoderN. The code uses a for...generate loop to assign values to the outputs based on the inputs and enable signal. The code is as follows:

```
1 library IEEE;
2 use IEEE.STD_LOGIC_1164.all;
3 use IEEE.NUMERIC_STD.all;
4
5 entity BinDecoderN is
6     generic(N : positive := 4);
7     port(enable : in std_logic;
8           inputs : in std_logic_vector(N-1 downto 0);
9           outputs : out std_logic_vector((2**N)-1 downto 0));
10 end BinDecoderN;
11
12 architecture BehavAssign of BinDecoderN is
13 begin
14     out_loop : for i in 0 to ((2**N) - 1) generate
15     --
16         outputs(i) <= enable when (i = to_integer(unsigned(inputs))) else
17             '0';
18     end generate;
19 end BehavAssign;
```

Exemplo de **for...generate** com um Processo

- Descodificador binário de $N \rightarrow 2^N$ bits, com N parametrizável

A screenshot of a text editor window titled "Text Editor - C:/Users/asoliveira/CloudStation/LSDig2017/SlidesTP/GenerateTest/Ge...". The editor displays VHDL code for a binary decoder. The code includes a library declaration, entity definition with generic and port parameters, and an architecture with a generate loop and a process block. The status bar at the bottom indicates "Ln 1 Col 1 VHDL File 100% 00:00:23".

```
1 library IEEE;
2 use IEEE.STD_LOGIC_1164.all;
3 use IEEE.NUMERIC_STD.all;
4
5 entity BinDecoderN is
6     generic(N : positive := 4);
7     port(enable : in std_logic;
8           inputs : in std_logic_vector(N-1 downto 0);
9           outputs : out std_logic_vector((2**N)-1 downto 0));
10 end BinDecoderN;
11
12 architecture BehavProc of BinDecoderN is
13 begin
14     out_loop : for i in 0 to ((2**N) - 1) generate
15     |
16     |   process(enable, inputs)
17     |   |   begin
18     |   |   |   if (i = to_integer(unsigned(inputs))) then
19     |   |   |   |   outputs(i) <= enable;
20     |   |   |   |
21     |   |   |   else
22     |   |   |   |   outputs(i) <= '0';
23     |   |   |   |   end if;
24     |   |   |   end process;
25     |   end generate;
26 end BehavProc;
27
```

Atributos Pré-definidos em VHDL

- Permitem testar condições e obter informação sobre tipos e objetos (*arrays*, sinais, portos, etc.) durante a simulação e a síntese de um modelo VHDL
- Vamos considerar apenas um pequeno subconjunto dos definidos na linguagem

Atributo aplicável a sinais e portos

- **SIGID'event** Event on signal
 - Exemplo: `if (clk'event and clk = '1')`

Atributos Aplicáveis a Tipos em VHDL

Úteis para escrever código mais flexível

<code>TYPID'left</code>	Left bound value
<code>TYPID'right</code>	Right-bound value
<code>TYPID'high</code>	Upper-bound value
<code>TYPID'low</code>	Lower-bound value
<code>TYPID'pos(expr)</code>	Position within type
<code>TYPID'val(expr)</code>	Value at position

Exemplos de Atributos Aplicáveis a Tipos em VHDL

```
integer'left    = -2147483648
integer'right   = 2147483647
integer'high    = 2147483647
integer'low     = -2147483648
integer'pos(0)  = 0
integer'val(0)  = 0
```

Considerando as declarações:

```
type std_ulogic is ('U', 'X', '0', '1', 'Z', 'W', 'L', 'H', '-');
subtype std_logic is resolved std_ulogic;
```

```
std_logic'left    = 'U'
std_logic'right   = '-'
std_logic'high    = '-'
std_logic'low     = 'U'
std_logic'pos('X') = 1    (valor inteiro)
std_logic'val(4)   = 'Z'
```

Atributos Aplicáveis a *Arrays* em VHDL

Úteis também para escrever código mais flexível

<code>ARYID[nth] 'left</code>	Left-bound of [nth] index
<code>ARYID[nth] 'right</code>	Right-bound of [nth] index
<code>ARYID[nth] 'high</code>	Upper-bound of [nth] index
<code>ARYID[nth] 'low</code>	Lower-bound of [nth] index
<code>ARYID[nth] 'range</code>	'left down/to 'right
<code>ARYID[nth] 'reverse_range</code>	'right down/to 'left
<code>ARYID[nth] 'length</code>	Length of [nth] dimension

Exemplos de Atributos Aplicáveis a *Arrays* em VHDL

Considerando as declarações:

```
subtype TDataWord is std_logic_vector(7 downto 0);  
type TMemory is array (0 to 31) of TDataWord;  
signal s_memory : TMemory;  
signal s_data : TDataWord;
```

```
s_memory'left      = 0  
s_memory'right     = 31  
s_memory'high      = 31  
s_memory'low       = 0  
s_memory'range     = (0 to 31)  
s_memory'reverse_range = (31 downto 0)  
s_memory'length    = 32
```

```
s_memory(0)'left      = 7  
s_memory(0)'right     = 0  
s_memory(0)'high      = 7  
s_memory(0)'low       = 0  
s_memory(0)'range     = (7 downto 0)  
s_memory(0)'reverse_range = (0 to 7)  
s_memory(0)'length    = 8
```

```
s_data'left      = 7  
s_data'right     = 0  
s_data'high      = 7  
s_data'low       = 0  
s_data'range     = (7 downto 0)  
s_data'reverse_range = (0 to 7)  
s_data'length    = 8
```



Comentários Finais

- No final desta aula deverá ser capaz de:
 - Utilizar as construções de VHDL
 - **for...generate**
 - **if...generate**
 - Conhecer alguns dos atributos pré-definidos de VHDL
 - Consultar em *reference cards* de VHDL as construções da linguagem abordadas em LSD